

Chapter 4

Stacks and Queues

Stacks and Queues

Basic operations

Implementation of stacks and queues

Example Applications

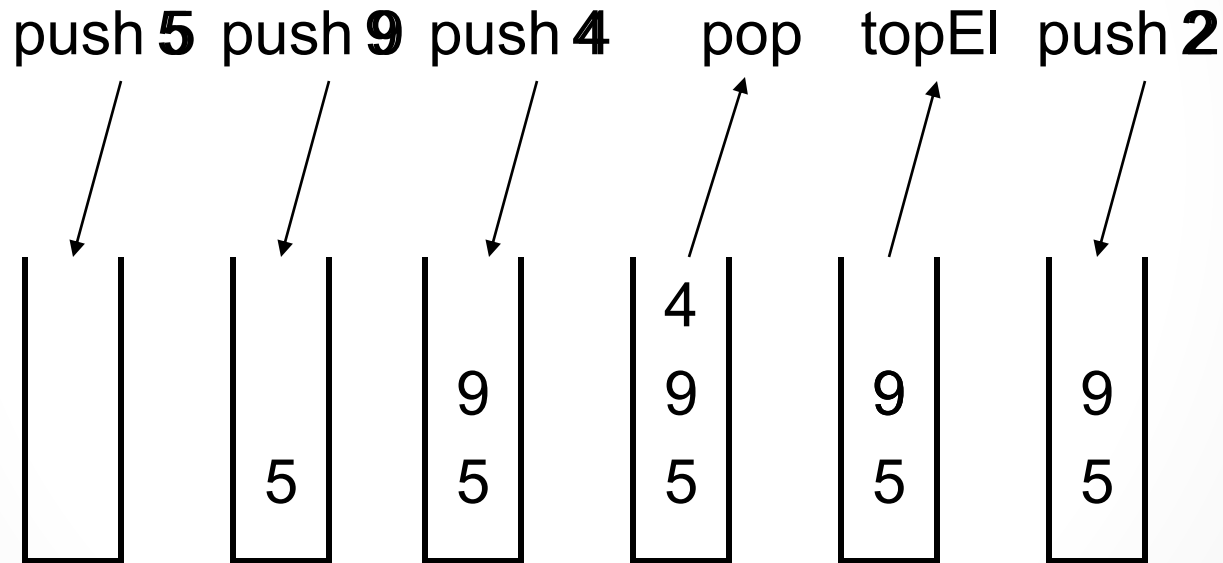
Stacks

- A **stack** is a linear data structure that can be accessed only at one of its ends for storing and retrieving data
- A stack is called an **LIFO** structure: last in/first out

Stack Operations

- push(el)** put the element el on the top of the stack;
- pop()** take the topmost element from the stack;
- topEl()** return the topmost element in the stack without removing it;
- clear()** clear the stack;
- isEmpty()** check to see if the stack is empty;

Series of Operations on a Stack



Applications of Stacks

- operations on long integers
- delimiter matching
- history of visited pages through a Web browser
- the undo sequence in a word processor
- sequence of method/function calls

Addition

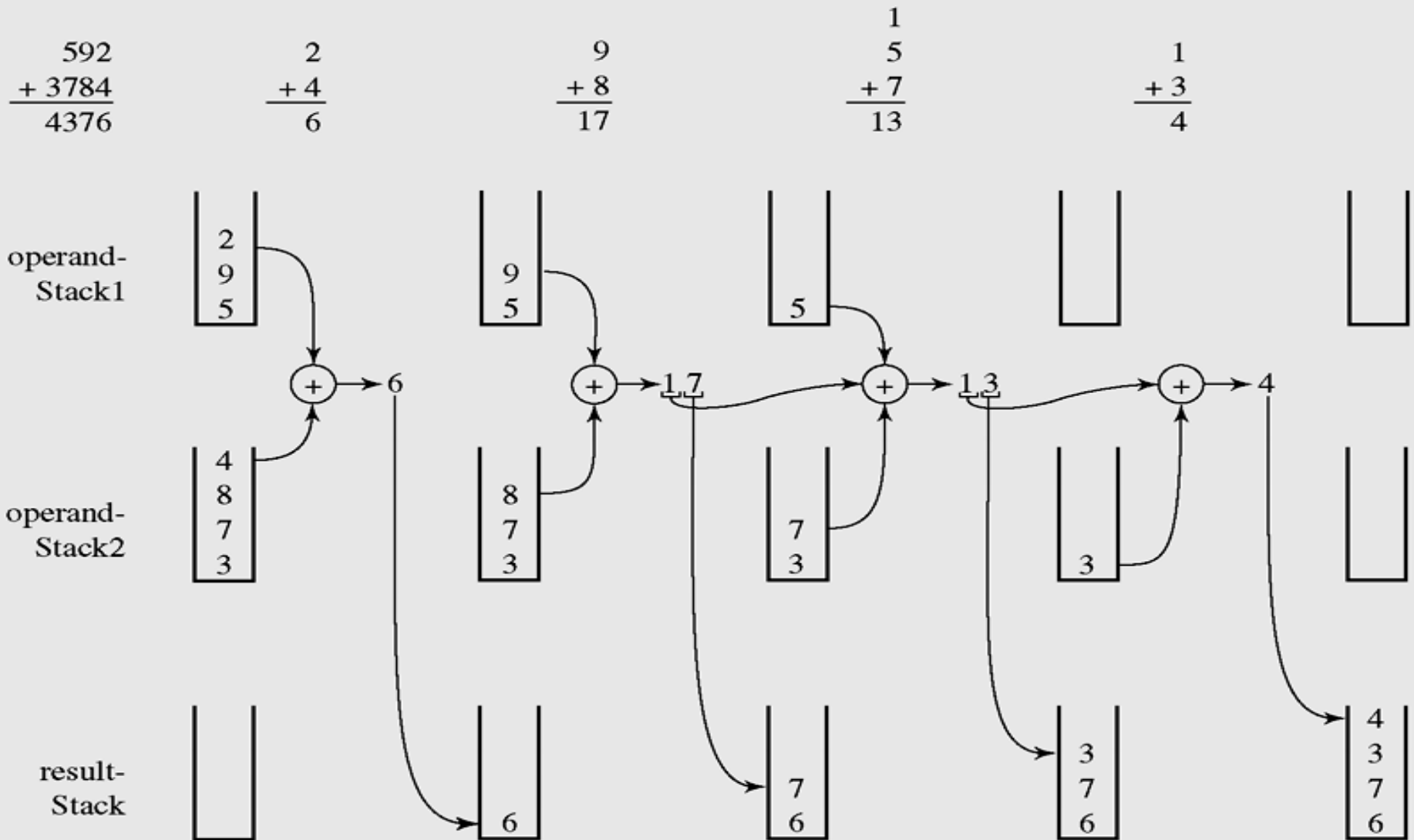


Figure 4-3 An example of adding numbers 592 and 3,784 using stacks

Array VS Linked List

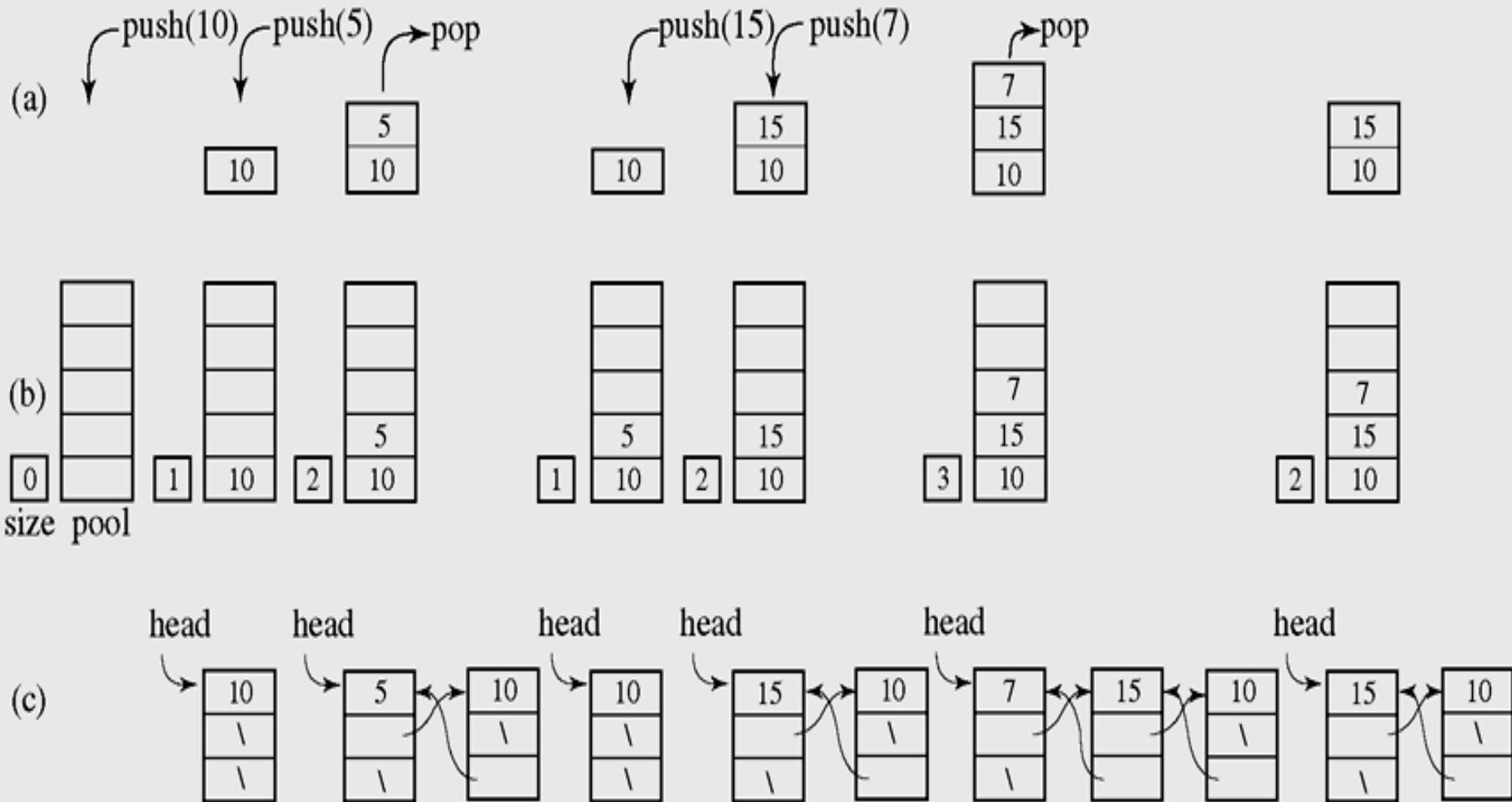


Figure 4-6 A series of operations executed on an abstract stack (a) and the stack implemented with an array (b) and with a linked list (c)

Queues

- A **queue** is a waiting line that grows by adding elements to its end and shrinks by taking elements from its front
- A queue is a structure in which both ends are used:
 - One for adding new elements
 - One for removing them
- A queue is an **FIFO** structure: first in/first out
-

Queue

enqueue(el) put the element el at the end of the queue;

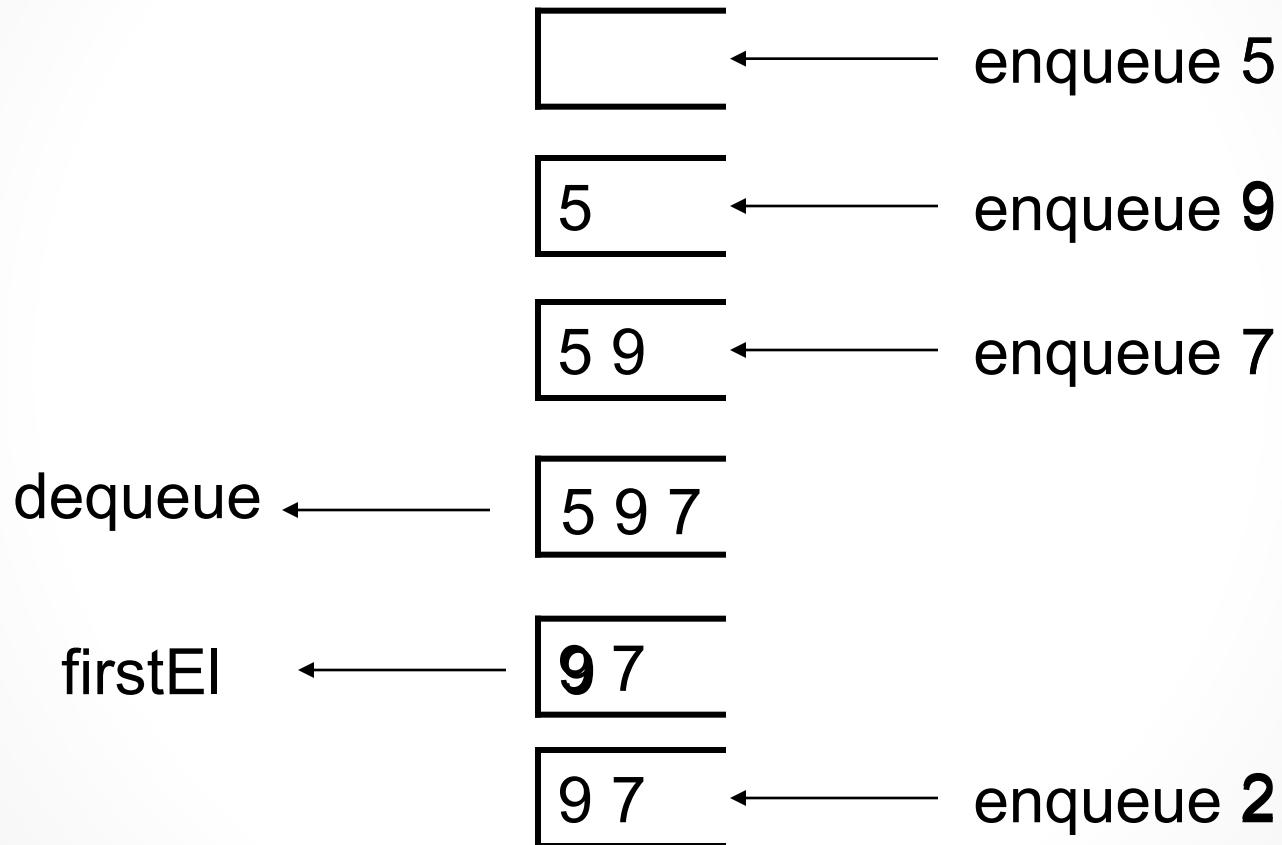
dequeue() take the first element from the queue;

firstEl() return the first element in the queue without removing it;

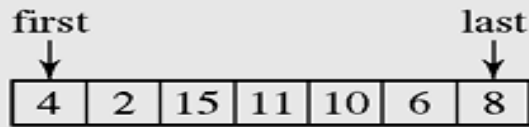
clear() clear the queue;

isEmpty() check to see if the queue is empty;

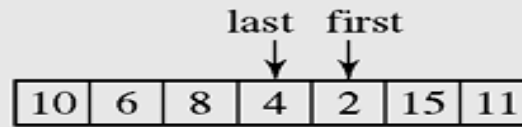
Series of Operations on a Queue



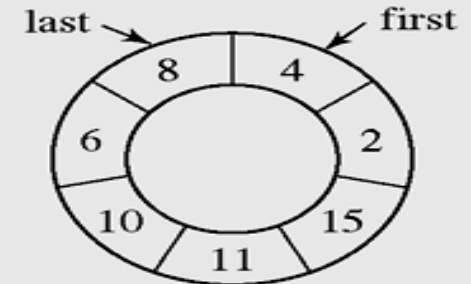
Handling Full Queue



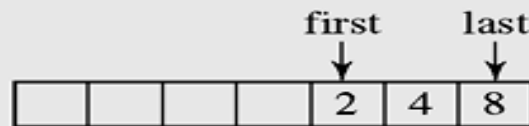
(a)



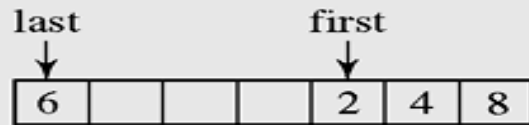
(b)



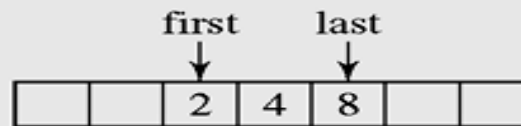
(c)



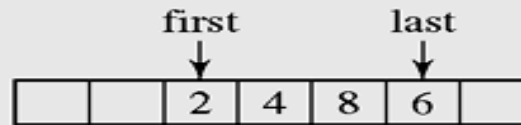
↓ enqueue(6)



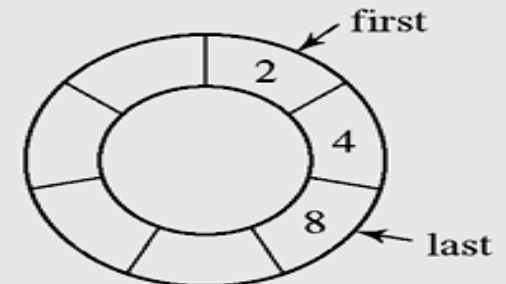
(d)



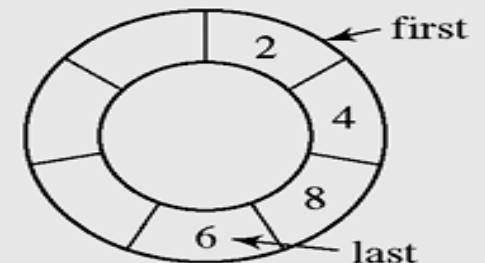
↓ enqueue(6)



(e)



↓ enqueue(6)



(f)

Figure 4-9 Two possible configurations in an array implementation of a queue when the queue is full

Queues using Array

```
public class ArrayQueue {
    private int first, last, size;
    private Object[] storage;
    public ArrayQueue() {
        this(100);
    }
    public ArrayQueue(int n) {
        size = n;
        storage = new Object[size];
        first = last = -1;
    }
    public boolean isFull() {
        return first == 0 && last == size-1 || first == last + 1;
    }
    public boolean isEmpty() {
        return first == -1;
    }
}
```

• **Figure 4-10 Array implementation of a queue**

Queues using Array (cont.)

```
}  
  
public void enqueue(Object el) {  
    if (last == size-1 || last == -1) {  
        storage[0] = el;  
        last = 0;  
        if (first == -1)  
            first = 0;  
    }  
    else storage[++last] = el;  
}
```

Queues using Array (cont.)

```
public Object dequeue() {  
    Object tmp = storage[first];  
    if (first == last)  
        last = first = -1;  
    else if (first == size-1)  
        first = 0;  
    else first++;  
    return tmp;  
}  
  
public void printAll() {  
    for (int i = 0; i < size; i++)  
        System.out.print(storage[i] + " ");  
}  
}
```

Queues (continued)

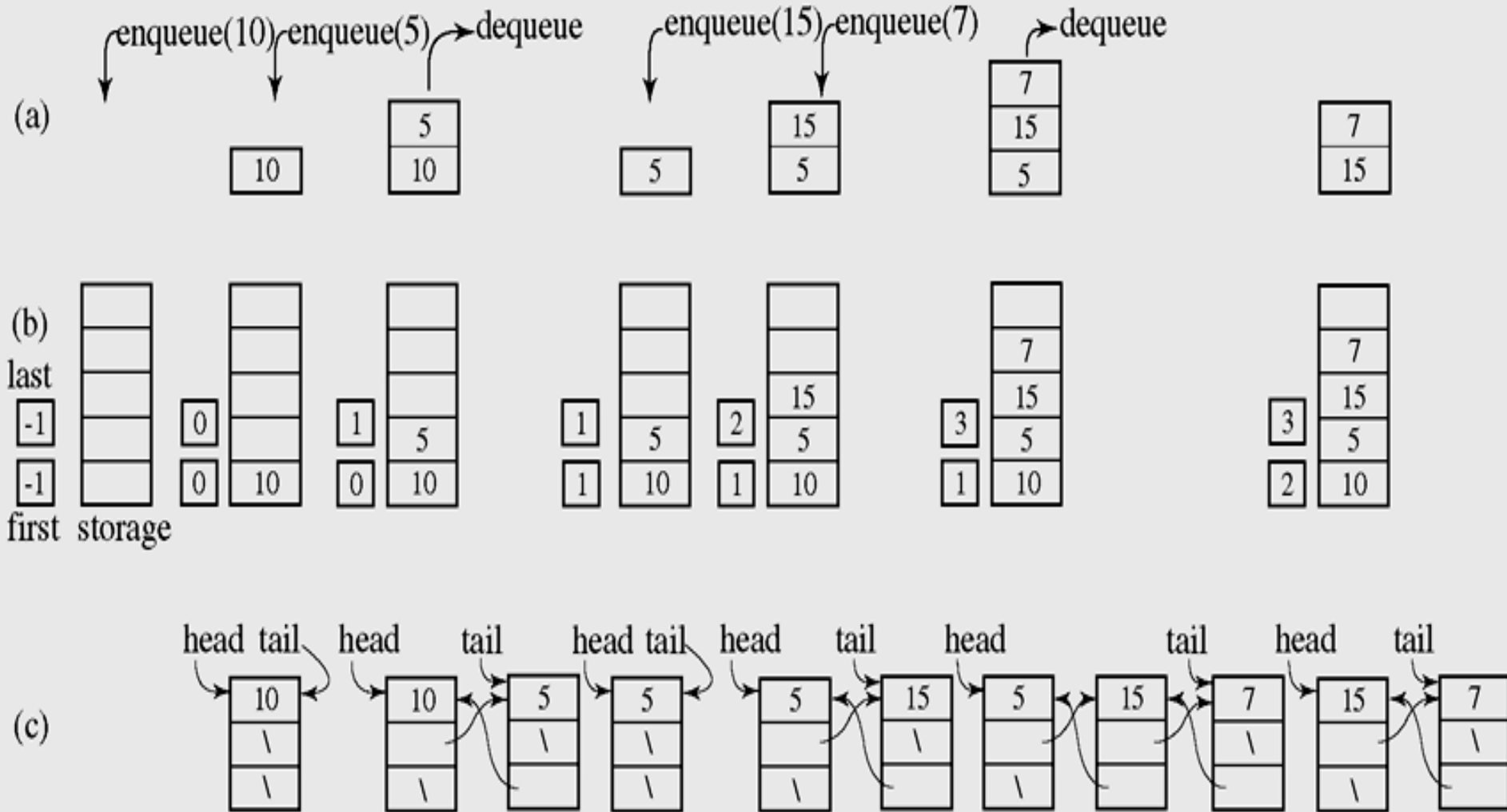


Figure 4-12 A series of operations executed on an abstract queue (a) and the stack implemented with an array (b) and with a linked list (c)

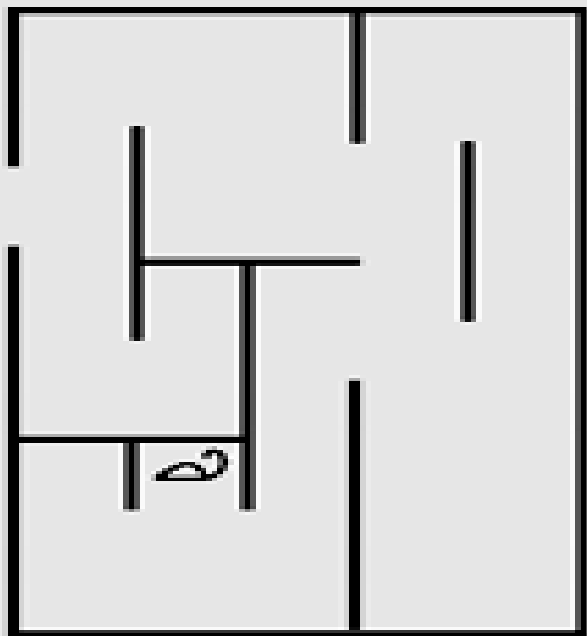
Priority Queues

- In **queuing theory**, various scenarios are analyzed and models are built that use queues for processing requests or other information in a predetermined sequence (order)
- A **priority queue** can be assigned to enable a particular process, or event, to be executed out of sequence without affecting overall system operation
- In priority queues, elements are dequeued according to their priority and their current queue position

Priority Queues (continued)

- Priority queues can be represented by two variations of linked lists:
 - All elements are entry ordered
 - Order is maintained by putting a new element in its proper position according to its priority

Case Study: Exiting a Maze



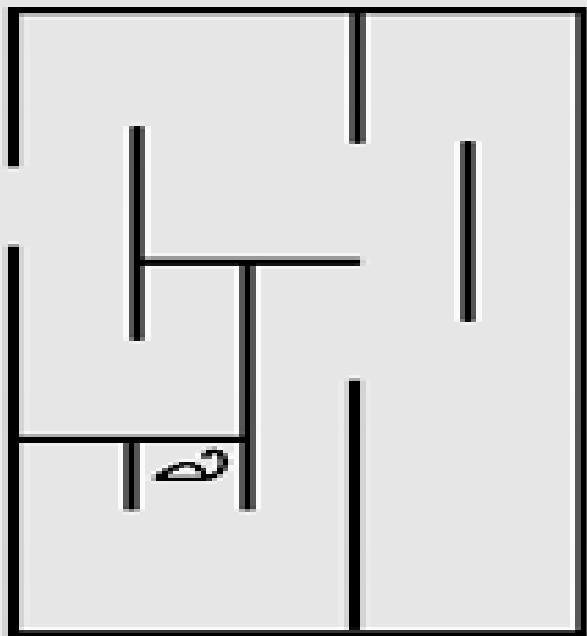
(a)

```
1 1 1 1 1 1 1 1 1 1 1
1 0 0 0 0 0 1 0 0 0 1
1 0 1 0 0 0 1 0 1 0 1
e 0 1 0 0 0 0 0 1 0 1
1 0 1 1 1 1 1 0 1 0 1
1 0 1 0 1 0 0 0 1 0 1
1 0 0 0 1 0 1 0 0 0 1
1 1 1 1 1 0 1 0 0 0 1
1 0 1 m 1 0 1 0 0 0 1
1 0 0 0 0 0 1 0 0 0 1
1 1 1 1 1 1 1 1 1 1 1
```

(b)

Figure 4-15 (a) A mouse in a maze; (b) two-dimensional character array representing this situation

Case Study: Exiting a Maze



(a)

```
1 1 1 1 1 1 1 1 1 1 1
1 0 0 0 0 0 1 0 0 0 1
1 0 1 0 0 0 1 0 1 0 1
e 0 1 0 0 0 0 0 1 0 1
1 0 1 1 1 1 1 0 1 0 1
1 0 1 0 1 0 0 0 1 0 1
1 0 0 0 1 0 1 0 0 0 1
1 1 1 1 1 0 1 0 0 0 1
1 0 1 m 1 0 1 0 0 0 1
1 0 0 0 0 0 1 0 0 0 1
1 1 1 1 1 1 1 1 1 1 1
```

(b)

Figure 4-15 (a) A mouse in a maze; (b) two-dimensional character array representing this situation

Case Study: Exiting a Maze (continued)

```
exitMaze()  
    initialize stack, exitCell, entryCell, currentCell = entryCell;  
while currentCell is not exitCell  
    mark currentCell as visited;  
    push onto the stack the unvisited neighbors of currentCell;  
    if stack is empty  
        failure;  
    else pop off a cell from the stack and make it currentCell;  
success;
```

Case Study: Exiting a Maze (continued)

stack:	(3 2) (2 3)	(3 1) (2 2) (2 3)	(2 1) (2 2) (2 3)	(2 2) (2 2) (2 3)	(2 3) (2 2) (2 3)	(2 4) (1 3) (2 2) (2 3)	(1 3) (2 2) (2 3)
currentCell:	(3 3)	(3 2)	(3 1)	(2 1)	(2 2)	(2 3)	(2 4)
maze:	111111 111001 1000e1 100m11 111111	111111 111001 1000e1 10.m11 111111	111111 111001 1000e1 1..m11 111111	111111 111001 1.00e1 1..m11 111111	111111 111001 1..0e1 1..m11 111111	111111 111001 1...e1 1..m11 111111	111111 111001 1...e1 1..m11 111111
	(a)	(b)	(c)	(d)	(e)	(f)	(g)

Figure 4-16 An example of processing a maze

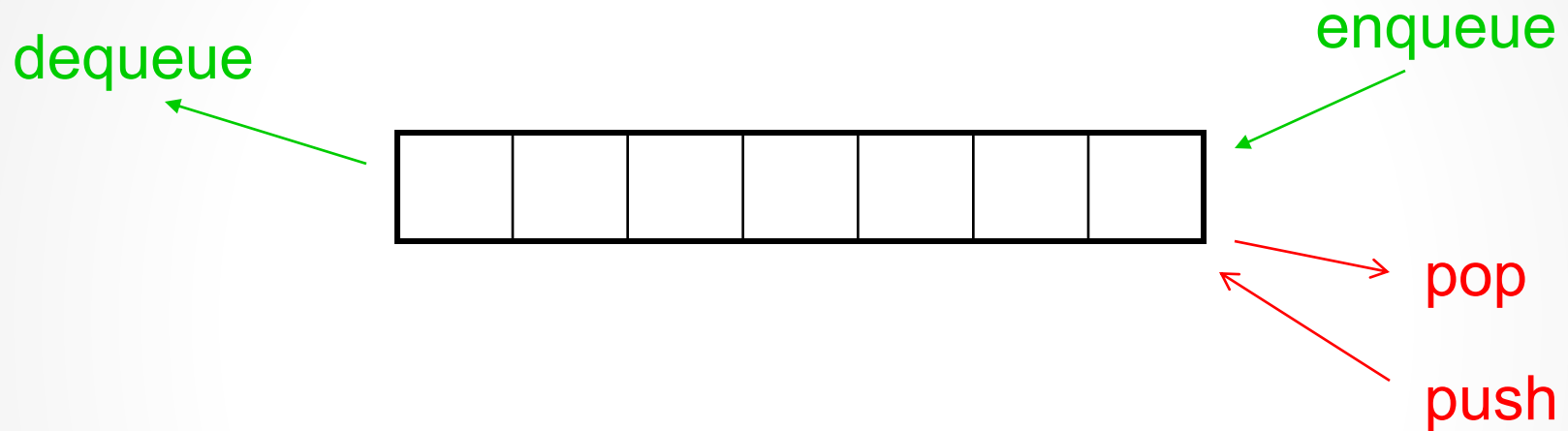
Applications of Queues

- waiting lines in simulation problems
- scheduling processes by the operating system
- access to shared resources

Summary

- A stack is a linear data structure that can be accessed at only one of its ends for storing and retrieving data.
- A stack is called an LIFO structure: last in/first out.
- A queue is a waiting line that grows by adding elements to its end and shrinks by taking elements from its front.
- A queue is an FIFO structure: first in/first out.

Stack vs. Queue



stack – a **LIFO** structure (last in, first out)

queue – a **FIFO** structure (first in, first out)

Summary (continued)

- In queuing theory, various scenarios are analyzed and models are built that use queues for processing requests or other information in a predetermined sequence (order).
- A priority queue can be assigned to enable a particular process, or event, to be executed out of sequence without affecting overall system operation.

Summary (continued)

- In priority queues, elements are dequeued according to their priority and their current queue position.

Reverse Stack Order

- Page 164
- Reverse the order of elements on stack S
 - using two additional stacks
 - using one additional queue
 - using one additional stack and some additional non-array variables

